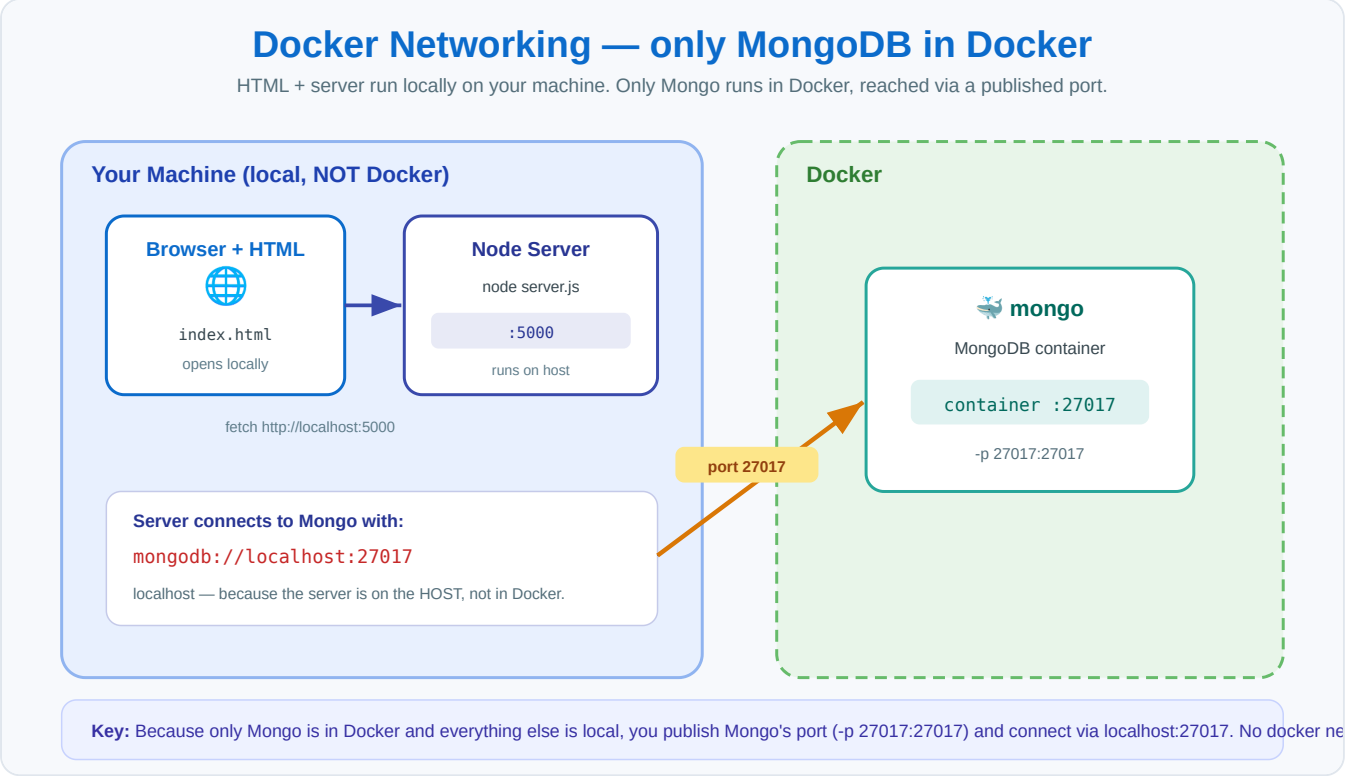
🐳 Docker Networking — Full Walkthrough (zero to running)

> HTML + server run **locally**; only **MongoDB** runs in Docker. Complete step-by-step from an empty folder to a working app.

Learner: Akshay • **Date:** 7 August 2026



Prerequisites: Docker Desktop running (green engine), Node.js installed.

Step 1 — Create the project folder

```
mkdir docker-net-demo
cd docker-net-demo
mkdir server
mkdir frontend
```

Structure:

```
docker-net-demo/
├── server/
└── frontend/
```

Step 2 — server/package.json

```
{
  "name": "docker-net-demo-server",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "mongodb": "^6.8.0"
  }
}
```

Step 3 — server/server.js

```
const express = require("express");
const { MongoClient } = require("mongodb");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());

// localhost — because THIS server runs on your machine, not in Docker.
// Mongo is reachable here because its container publishes -p 27017:27017
const MONGO_URL = "mongodb://localhost:27017";
const PORT = 5000;

let db;

async function start() {
  const client = new MongoClient(MONGO_URL);
  await client.connect();
  db = client.db("demo");
  console.log("Connected to MongoDB");
  app.listen(PORT, () => console.log("Server on http://localhost:" + PORT));
}

app.get("/api/health", (req, res) => res.json({ status: "ok" }));

app.get("/api/messages", async (req, res) => {
  const messages = await db.collection("messages").find().toArray();
  res.json(messages);
});

app.post("/api/messages", async (req, res) => {
  const result = await db.collection("messages").insertOne({ text: req.body.text });
  res.json({ _id: result.insertedId, text: req.body.text });
});

start().catch(err => { console.error(err); process.exit(1); });
```

Step 4 — Install server dependencies

```
cd server
npm install
```

Creates `node_modules/` with Express, the MongoDB driver, and CORS. Run this from inside `server/`.

Step 5 — frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Docker Network Demo</title>
  <style>
    body { font-family: 'Segoe UI', Arial, sans-serif; max-width: 600px; margin: 40px auto; }
    input { padding: 10px; width: 70%; }
    button { padding: 10px 16px; background: #0b6bcb; color: #fff; border: none; border-radius: 6px
  </style>
</head>
<body>
  <h1>Docker Network Demo</h1>
  <input id="msg" placeholder="Type a message..." />
  <button onclick="addMessage()">Add</button>
  <ul id="list"></ul>
  <p id="status">Loading...</p>

  <script>
    const API = "http://localhost:5000";   // the LOCAL server

    async function load() {
      try {
        const res = await fetch(API + "/api/messages");
        const data = await res.json();
        document.getElementById("list").innerHTML = data.map(m => "<li>" + m.text + "</li>").join("
        document.getElementById("status").textContent = "Connected to server + MongoDB";
      } catch {
        document.getElementById("status").textContent = "Cannot reach server on :5000";
      }
    }
    async function addMessage() {
      const text = document.getElementById("msg").value;
      if (!text) return;
      await fetch(API + "/api/messages", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text })
      });
      document.getElementById("msg").value = "";
      load();
    }
    load();
  </script>
</body>
</html>
```

Step 6 — Start MongoDB in Docker (the only Docker part)

```
docker run -d --name mongo -p 27017:27017 mongo
```

`-d` background, `--name mongo` names it, **`-p 27017:27017` publishes the port** — the door that lets your local server reach the DB at `localhost:27017`.

Verify:

```
docker ps
```

Step 7 — Start the server (locally)

From `server/`:

```
node server.js
```

Expected output:

```
Connected to MongoDB
Server on http://localhost:5000
```

Leave this terminal running.

Step 8 — Open the frontend

Double-click `frontend/index.html` (or drag it into your browser). You should see "Connected to server + MongoDB".

Step 9 — Test the full chain

Type a message → Add. It flows: **Browser → local server (:5000) → MongoDB container (:27017)** and is saved. Refresh — the message reloads from the Dockerized DB. End-to-end works.

Step 10 — Cleanup

```
# stop the server: Ctrl+C in its terminal
docker stop mongo
docker rm mongo
```

> ⚠️ `docker rm mongo` deletes the data (it lived in the container's writable layer). To keep data between runs, use a volume: `bash docker run -d --name mongo -p 27017:27017 -v mongo-data:/data/db mongo`

Quick recap of the flow

- Only **MongoDB** is in Docker; **HTML + server** run locally.
- The server connects with `mongodb://localhost:27017` — works only because Mongo published its port with `-p 27017:27017`.
- The browser calls the local server at `http://localhost:5000`.
- No `docker network` needed, because just one thing is containerized.